Manual de Aplicación Web y Base de Datos SQLite

Integrantes: Pepe y Chera

PARTE 1: MANUAL DE USUARIO

1. ¿Para qué sirve esta página?

Esta aplicación web te permite ingresar tres datos principales:

- Nombre (Texto)
- Valor (Número/Monto)
- Fecha (Día, mes y año)

Al tocar el botón de guardado, la información se envía mediante una API a un servidor Node.js, donde queda grabada de forma permanente en un archivo de base de datos SQLite (`database.db`). Los datos no se pierden al cerrar o actualizar la página y se muestran inmediatamente en una lista en tiempo real.

2. Pasos para usar la aplicación

1. Iniciar el Servidor:
    - Abre la terminal en la carpeta del proyecto.
    - Ejecuta el comando `node server.js` para iniciar la base de datos y el servidor.
2. Abrir la Aplicación:
    - Abre el archivo `index.html` haciendo doble clic desde cualquier navegador web (Google Chrome, Firefox, Edge, etc.).
3. Llenar los campos:
    - En la casilla Nombre, escribe el nombre de la persona, producto o registro.
    - En la casilla Valor, ingresa un número o precio (admite decimales).
    - En la casilla Fecha, selecciona la fecha deseada usando el calendario desplegable.
4. Guardar en la Base de Datos:
    - Haz clic en el botón Guardar en Servidor.
    - Si dejas algún campo vacío, el navegador te avisará que debes completarlo.
5. Ver los registros guardados:
    - Apenas hagas clic en guardar, los campos se limpiarán automáticamente para cargar un nuevo registro.
    - Abajo, en la sección "Datos guardados en la BD del Servidor", verás tu nuevo registro publicado en una lista ordenada.

PARTE 2: MANUAL TÉCNICO

1. Explicación de la Arquitectura y Conexión

¿Cómo se conecta HTML con SQLite?

HTML por sí solo no puede conectar con un archivo de base de datos SQLite en disco de forma directa por razones de seguridad del navegador. Por ello, implementamos una arquitectura Cliente-Servidor:

1. Frontend (HTML/JavaScript): Se encarga de la interfaz y captura de eventos.
2. Backend (Node.js/Express): Servidor local que escucha peticiones HTTP en el puerto `3000`.
3. Base de Datos (SQLite3): Motor de base de datos relacional ligero que almacena los datos en la raíz del proyecto en un archivo llamado `database.db`.

El circuito de la información funciona así:

1. Captura: El usuario llena los inputs en `index.html` y presiona el botón de envío.
2. Intercepción y Petición HTTP: JavaScript detiene el refresco de la página (`e.preventDefault()`) y hace un envío asíncrono con `fetch()` mediante la ruta `POST http://localhost:3000/api/datos`.
3. Procesamiento Backend: El servidor Express recibe los datos en formato JSON y ejecuta una consulta SQL parametrizada (`INSERT INTO datos`).
4. Respuesta y Actualización: Al recibir confirmación exitosa del servidor, JavaScript realiza una petición `GET http://localhost:3000/api/datos` para solicitar la lista actualizada y dibujarla en el HTML.

2. Código Fuente Completo

A. Backend (`server.js`)

JavaScript

```
const express = require('express');

const sqlite3 = require('sqlite3').verbose();

const cors = require('cors');



const app = express();

const PORT = 3000;



// Middlewares

app.use(express.json());
```

```
app.use(cors());

// Conexión a la base de datos SQLite persistente
const db = new sqlite3.Database('./database.db', (err) => {
    if (err) {
        console.error('Error al conectar con SQLite:', err.message);
    } else {
        console.log('Conectado a la base de datos SQLite persistente.');
    }
});

// Crear tabla si no existe
db.run(`CREATE TABLE IF NOT EXISTS datos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT,
    valor REAL,
    fecha TEXT
)`);

// Ruta POST: Insertar un registro
app.post('/api/datos', (req, res) => {
    const { nombre, valor, fecha } = req.body;
    const query = `INSERT INTO datos (nombre, valor, fecha) VALUES (?, ?, ?)`;

    db.run(query, [nombre, valor, fecha], function(err) {
        if (err) {
```

```
            return res.status(500).json({ error: err.message });
        }
        res.json({ message: 'Registrado con éxito', id: this.lastID });
    });
});

// Ruta GET: Obtener todos los registros
app.get('/api/datos', (req, res) => {
    const query = `SELECT nombre, valor, fecha FROM datos`;
    db.all(query, [], (err, rows) => {
        if (err) {
            return res.status(500).json({ error: err.message });
        }
        res.json(rows);
    });
});

// Iniciar servidor
app.listen(PORT, () => {
    console.log(`Servidor corriendo en http://localhost:${PORT}`);
});
```

B. Frontend (`index.html`)

HTML

```
<!DOCTYPE html>
<html lang="es">
```

```
<head>
  <meta charset="UTF-8">
  <title>Formulario SQLite</title>
</head>
<body>

  <h2>Registrar Datos</h2>

  <!-- Formulario principal -->
  <form id="miFormulario">
    <label>Nombre:</label><br>
    <input type="text" id="nombre" required><br><br>

    <label>Valor:</label><br>
    <input type="number" step="0.01" id="valor" required><br><br>

    <label>Fecha:</label><br>
    <input type="date" id="fecha" required><br><br>

    <button type="submit">Guardar en Servidor</button>
  </form>

  <h3>Datos guardados en la BD del Servidor</h3>
  <ul id="listaRegistros"></ul>

  <!-- Lógica y llamadas a la API REST -->
```

```
  <script>
    const API_URL = 'http://localhost:3000/api/datos';

    // Cargar los registros guardados al abrir la página
    window.onload = mostrarRegistros;

    // Evento del formulario al enviar
    document.getElementById('miFormulario').addEventListener('submit', async
function(e) {
      e.preventDefault();

      const nombre = document.getElementById('nombre').value;
      const valor = parseFloat(document.getElementById('valor').value);
      const fecha = document.getElementById('fecha').value;

      try {
        const response = await fetch(API_URL, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ nombre, valor, fecha })
        });

        if (response.ok) {
          this.reset();
          mostrarRegistros();
        } else {
          alert('Error al guardar el registro en el servidor.');
```

```
      }
    } catch (error) {
      console.error('Error de red:', error);
    }
  });

  // Función para consultar y mostrar los registros en el HTML
  async function mostrarRegistros() {
    const lista = document.getElementById('listaRegistros');
    lista.innerHTML = "";

    try {
      const response = await fetch(API_URL);
      const filas = await response.json();

      filas.forEach(row => {
        const li = document.createElement('li');
        li.textContent = `Nombre: ${row.nombre} | Valor: ${row.valor} | Fecha: ${row.fecha}`;
        lista.appendChild(li);
      });
    } catch (error) {
      console.error('Error al cargar registros:', error);
    }
  }
</script>
```

```
</body>

</html>
```

3. Explicación del Código Paso a Paso

A. Backend (`server.js`)

1. Configuración de Express y CORS: Se activa `cors()` para permitir que la página HTML acceda al servidor aunque esté abierta desde un archivo local o servidor distinto.
2. Conexión a la Base de Datos: `new sqlite3.Database('./database.db')` crea o se conecta automáticamente al archivo físico `database.db`.
3. Definición de Tabla: La sentencia `CREATE TABLE IF NOT EXISTS datos` asegura que las columnas `id`, `nombre`, `valor` y `fecha` estén disponibles desde la primera ejecución.
4. Ruta `POST /api/datos`: Utiliza consultas preparadas con el marcador ? para prevenir inyecciones SQL y guarda el nombre, valor y fecha enviados desde el cliente.
5. Ruta `GET /api/datos`: Ejecuta un `SELECT` en SQLite para recuperar todas las filas de la base de datos y responder al frontend en formato JSON.

B. Frontend (`index.html`)

1. Atributo `window.onload = mostrarRegistros`: Ejecuta la lectura inicial de datos apenas se termina de cargar la página en el navegador.
2. Método `fetch(API_URL, { method: 'POST', ... })`: Envía un paquete HTTP con los valores digitados por el usuario serializados en un objeto JSON hacia Node.js.
3. Método `fetch(API_URL)`: Consume la API en modo `GET`, parsea los registros devueltos y utiliza el DOM de JavaScript para crear etiquetas `<li>` dinámicamente dentro de `<ul id="listaRegistros">`.